

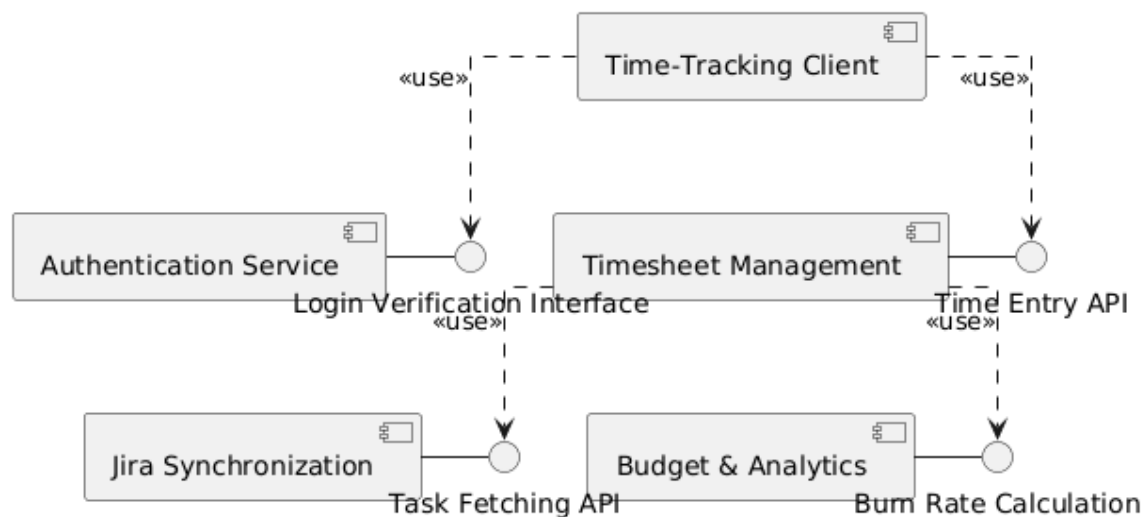
SE Lab 3: Component Modelling & Architectural Pattern Selection

Name: Suhas Mohan Kumar

SRN: PES1UG24AM290

Problem Statement: Remote Team Time-Tracking & Project Approver(46)

Deliverable 1: Component Diagram & Interface Specifications



Component List:

- 1. Time-Tracking Client (User interface for developers and managers)
- 2. Authentication Service (Manages secure logins and session tokens)
- 3. Timesheet Management Component (Validates daily 24-hour limits and processes submissions)
- 4. Jira Synchronization Component (Automatically fetches active tasks assigned to developers)
- 5. Budget & Analytics Component (Calculates real-time project budget burn rates)

Interface Summary Table:

Interface Name	Connection / Flow	Purpose
Login Verification	Time-Tracking Client → Authentication Service	Secures user authentication and login sessions
Time Entry API	Time-Tracking Client → Timesheet Management	Sends logged hours and enforces the 24-hour daily limit rule
Task Fetching API	Timesheet Management → Jira Synchronization	Fetches and synchronizes active Jira tickets for the developer
Burn Rate Calculation	Timesheet Management → Budget & Analytics	Supplies approved hours to calculate and update real-time burn rates

Deliverable 2: Architecture Analysis & Written Justification

Architecture Selection: We chose Microservices Architecture for the Remote Team Time-Tracking & Project Approver System.

Architectural Style Analysis:

- Layered Architecture: Keeps responsibilities separated, but passing continuous time-tracking updates through multiple layers adds processing overhead and complicates independent scaling.
- Microservices Architecture: Separates tracking, timesheets, authentication, Jira integration, and analytics into independent services that can scale separately under heavy usage loads [cite: 4].
- Client-Server Architecture: Simple to deploy and centralizes data, but a single central server can become a bottleneck and single point of failure if multiple distributed remote developers log time concurrently [cite: 4].

Architectural Choice & Reasons:

- The Microservices architecture allows independent modules to interact seamlessly, which is essential for integrating internal time-logging mechanisms with external systems like Jira for real-time task synchronization.
- It supports targeted scalability, ensuring that high-demand features like real-time project budget burn rate calculations and dashboard sync operate concurrently without performance bottlenecks.

Security Advantage:

- By isolating authentication into its own dedicated service, the system ensures that secure login processes for both remote developers and Engineering Managers are centralized.
- This prevents vulnerabilities in the timesheet entry forms or external Jira integrations from compromising user credentials or manager approval workflows.

Performance Benefit & Fault Isolation:

- Microservices offer strong fault isolation, directly supporting non-functional requirements like 99.9% uptime and <500ms latency [cite: 2].
- If the Jira synchronization service encounters temporary downtime, remote developers can still securely authenticate and log time against cached tasks, preventing total system failure.